

DIO EBOOK • EDIÇÃO JEDI

TYPESCRIPT

O CAMINHO JEDI

Guia Prático de Tipagem Estrita,
Generics e Arquitetura Limpa



TS

Sem Any

Generics

Utility Types

SOLID

Autor: Matheus Araujo (@MatheusMittz)

Formação ChatGPT para Devs | Digital Innovation One

Apresentação & Filosofia Jedi

No universo do desenvolvimento de software, a pressa e a busca por atalhos fáceis muitas vezes conduzem os desenvolvedores ao que chamamos de **O Lado Sombrio do Código**. No ecossistema TypeScript, a tentação mais sedutora é o uso indiscriminado do `any`. Ele promete liberdade instantânea, mas entrega caos, bugs silenciosos e manutenção penosa.

Este eBook foi estruturado como um guia prático para transformar você em um verdadeiro **Mestre Jedi do TypeScript**. Aqui você aprenderá a forjar tipos robustos utilizando **Generics**, manipular estruturas com os 4 **Utility Types** essenciais e estruturar camadas limpas de DTOs e Repositórios inspiradas nos princípios SOLID.

Sumário Galáctico

Capítulo 1: O Lado Sombrio do `any` (E por que evitá-lo a todo custo)

Capítulo 2: O Sabre de Luz dos Generics (Tipos reutilizáveis e escaláveis)

Capítulo 3: Os 4 Holocrons dos Utility Types (Partial, Pick, Omit e Record)

Capítulo 4: A Força da Arquitetura Limpa (DTOs seguros sem duplicação)

Capítulo 5: O Conselho Jedi: Checklist de Boas Práticas e Conclusão

Capítulo 1: O Lado Sombrio do any

Quando tipamos uma variável com `any`, estamos dizendo explicitamente ao compilador do TypeScript: *'Não analise este valor. Confie cegamente em mim.'* O problema é que humanos falham. Funções renomeadas continuam compilando, parâmetros nulos passam despercebidos e o erro só explode quando o cliente final tenta clicar em um botão em produção.

A Alternativa Luminosa: `unknown` e Type Guards

Ao contrário do `any`, o tipo `unknown` representa um valor de natureza incerta, mas **obriga** o desenvolvedor a realizar uma verificação (Type Guard) antes de acessar qualquer propriedade:

```
// Inseguro: compila, mas pode quebrar em runtime
function unsafeLog(data: any) { console.log(data.user.name); }

// Seguro com unknown + Type Narrowing
function safeLog(data: unknown) {
  if (typeof data === 'object' && data !== null && 'name' in data) {
    console.log((data as { name: string }).name);
  }
}
```

Capítulo 2: O Sabre de Luz dos Generics

Generics são como variáveis de tipo. Eles permitem que funções, classes e interfaces recebam tipos como argumentos, garantindo total flexibilidade sem jamais sacrificar a verificação estática.

Envelope Padronizado de Resposta HTTP

```
export interface ApiResponse<TData> {
  statusCode: number;
  success: boolean;
  timestamp: string;
  data: TData;
  error?: string;
}

// O compilador deduz exatamente as propriedades do usuário:
const userResponse: ApiResponse<{ id: string; name: string }> = {
  statusCode: 200,
  success: true,
  timestamp: new Date().toISOString(),
  data: { id: 'u-1', name: 'Luke Skywalker' }
};
```

Capítulo 3: Os 4 Holocrons dos Utility Types

Os Utility Types são transformadores nativos que geram novos tipos a partir de interfaces existentes, eliminando o terrível antipadrão de duplicar campos manualmente.

Utility Type	Propósito Jedi	Cenário de Aplicação
Partial <T>	Torna todos os atributos opcionais (?).	Payloads de PATCH e filtros de pesquisa.
Pick <T, K>	Extraí apenas o subconjunto de chaves K.	Resumos de listagem e tabelas simplificadas.
Omit <T, K>	Remove estritamente os campos sensíveis K.	Remoção de senhas e auditorias da resposta.
Record <K, T>	Dicionário chave-valor estritamente tipado.	Mapas de permissões e matrizes de estado.

Capítulo 4: A Força da Arquitetura Limpa

Na Arquitetura Limpa, as entidades de domínio representam a verdade central do sistema. Combinando `Omit` e `Partial`, criamos contratos de entrada (DTOs) que derivam organicamente da entidade sem jamais duplicá-la:

```
export interface Order {
  id: string;
  customerId: string;
  total: number;
  status: 'PENDING' | 'PAID' | 'CANCELED';
  createdAt: Date;
}

// Criação: Não possui ID nem createdAt (gerados pelo banco)
export type CreateOrderDto = Omit<Order, 'id' | 'createdAt'>;

// Atualização: Atualiza status ou total de forma parcial
export type UpdateOrderDto = Partial<Pick<Order, 'status' | 'total'>>;
```

Capítulo 5: O Conselho Jedi & Conclusão

Checklist de Qualidade para Todo Desenvolvedor

1. **Ative o modo estrito** no tsconfig.json ("strict": true).
2. **Banir o any** do projeto utilizando regras de lint (@typescript-eslint/no-explicit-any).
3. **Centralize DTOs** derivados de entidades mestras com Pick e Omit.
4. **Documente seus tipos** com JSDoc para proporcionar DX (Developer Experience) impecável.

Agradecimentos e Conexão

Parabéns por completar esta jornada! O domínio da tipagem estrita é o divisor de águas entre quem apenas escreve scripts e quem constrói sistemas corporativos resilientes.

Autor: Matheus Araujo (MatheusMittz)

GitHub: <https://github.com/MatheusMittz>

Comunidade: Formação ChatGPT para Devs na Digital Innovation One (DIO)

Instrutor de Referência: Felipe Aguiar